Industrial PPE, Smoke & Fire Safety Compliance AI System

Complete project plan, feature scope, do's and don'ts, alternatives, work structure and day-by-day schedule

Hackathon Problem Statement 6 (PS6) | 10-12 days

Prepared for: Hackathon team

Problem statement: PS6 - AI system for safety gear compliance, smoke and fire detection with located alerts

Time available: 10-12 days

How to use this document

- Sections 1-5: what the company needs, what to build, what to avoid, and the architecture.
- Sections 6-9: technology choices with alternatives, detailed module design, data/model plan, robustness.
- Sections 10-12: who does what, the day-by-day schedule (12 and 10 day versions), testing.
- Sections 13-17: deployment/cost, privacy, risks, pitch, judging checklist. Appendices: repo structure, Day-1 checklist, items to verify, and the target dashboard design.

Contents

1. Executive summary . . . 3
2. What the company (and the judges) need . . . 3
3. Scope: what to build (three tiers) . . . 4
4. Do's and Don'ts . . . 5
5. System architecture . . . 6
6. Technology choices and alternatives . . . 6
7. Detailed module design . . . 7
8. Data and model plan . . . 10
9. Robustness plan and failure-case gallery . . . 11
10. Team structure and ways of working . . . 11
11. Schedule (12 days, with a 10-day version) . . . 13
12. Testing and evaluation plan . . . 14
13. Deployment, bandwidth and cost angle . . . 15
14. Privacy, ethics and honest limitations . . . 16
15. Risk register . . . 16
16. Demo and pitch plan . . . 16
17. Judging checklist mapping . . . 17
Appendix A. Suggested repository structure . . . 18
Appendix B. Day-1 checklist . . . 18
Appendix C. Things to verify yourselves . . . 18
Appendix D. Dashboard design reference (SentinelEye) . . . 19

1. Executive summary

What we are building. A prototype, edge-capable safety intelligence system that watches CCTV or recorded factory video and (a) checks whether each worker wears the PPE required for the zone they stand in, and (b) detects early smoke and fire. Every event becomes a located, prioritised alert (zone, camera, type, severity, time, snapshot, confidence) that a floor supervisor can acknowledge, escalate or dismiss.

What makes it different from the typical hackathon demo. Most teams will show a pretrained YOLO model drawing a box and printing 'No helmet'. Our edge comes from the layers around the model:

- **Per-person, per-item compliance** ('worker 7: helmet missing, gloves unverifiable'), not a binary violation flag.
- **Zone-aware rules:** welding bay needs goggles + gloves, warehouse only helmet + vest.
- **Tracking + temporal logic:** one alert per violation event, with a grace period, instead of alert spam.
- **Severity tiers:** fire is not the same as a missing glove, and the system behaves accordingly.
- **Documented robustness:** dust, steam, low light, oblique angles, occlusion, and the 'orange vest is not fire' false-positive case.
- **Edge-first** with a stated latency, bandwidth and cost justification, plus multi-stream health handling.

How the plan is organised. The work is split into workstreams A (ML/vision), B (pipeline/alert logic) and C (backend + dashboard) that run in parallel against frozen interface contracts. Two integration milestones (end of Day 5 and Day 9), a feature freeze at Day 10, and a buffer at the end. A 10-day compressed version is included.

2. What the company (and the judges) need

The end user is a **floor safety supervisor**, not a data scientist. The problem statement and your PPE documentation translate into the following needs. We build to this table.

Company need	Why it matters in a real factory	How we deliver it
Know which PPE item is missing, for which worker	A binary 'violation' gives the supervisor nothing to act on.	Person + item detection, geometry-based association, per-item status (compliant / missing / unverifiable).
Different rules per area	A welding bay and a warehouse do not have the same PPE requirements ('gloves where applicable').	Zone polygons per camera + YAML rules; the same model serves every zone.
Detect smoke and fire early	Minutes matter for evacuation and damage.	Dedicated fire/smoke detector + growth tracking + two-stage verification.
Fire is treated as more urgent than a missing glove	Otherwise supervisors cannot triage.	4-tier severity engine with escalation rules and different response channels.
Alerts that carry location and evidence	'Something happened somewhere' slows response.	Alert card: zone, camera ID, event type, timestamp, snapshot, confidence.
Very few false alarms	Alert fatigue makes staff ignore the system.	N-of-M frame confirmation, grace period, cooldown, fire look-alike suppression, confidence display.
Works in dust, steam, dark, odd angles, crowds	Factories are not clean lab footage.	Augmented training data, hard test set, failure-case gallery, 'unverifiable' state for occluded workers.
Many cameras, not one	A plant has dozens of feeds.	Multi-stream manager with per-stream health and load-aware frame skipping.
No raw video leaving the site	Bandwidth, cost, latency, privacy.	Edge inference; only metadata, alerts and thumbnails go to the dashboard.
Daily usability and accountability	The tool must fit the supervisor's shift.	Acknowledge/escalate/dismiss workflow, shift/zone reports (CSV/PDF), trend analytics.
Affordable and deployable	Plants buy on ROI.	Cost/ROI framework, hardware options, honest limitations.

3. Scope: what to build (three tiers)

Tier 1 (Must) is required to satisfy the brief and the judging checklist. Tier 2 (Should) is what separates a good project from an ordinary one. Tier 3 (Wow) is only started once Tier 1 is stable and Tier 2 is done or consciously dropped. Each feature has an owner, a target window and a Plan B so that a failure never blocks the demo.

ID	Feature	Tier	Owner	Days	Plan B if it goes wrong
F01	PPE detection: person, helmet, vest, boots, gloves (+ goggles/mask if the dataset supports it)	Must	A	1-8	Drop goggles/mask; keep helmet + vest + boots; treat gloves as advisory (low-confidence) only.
F02	PPE-to-person association with per-item missing output and an 'unverifiable' state	Must	B	3-6	Train explicit 'no_helmet' style classes on a small set instead of geometry rules.
F03	Fire and smoke detection	Must	A	3-8	Use a public pretrained fire/smoke model and put the effort into the verification layer (F13).
F04	Multi-object tracking (ByteTrack via the Supervision library)	Must	B	2-5	Simple IoU tracker written by hand.
F05	Temporal smoothing (N of M frames), grace period, dedup, cooldown	Must	B	3-6	Fixed 'N consecutive frames' rule.
F06	Zone rules engine (polygons + YAML requirements per zone)	Must	B	2-5	Rectangular zones in a config file; no drawing UI.
F07	Severity engine (Critical / High / Medium / Low)	Must	B	4-6	Static severity table keyed by event type and zone risk.
F08	Multi-stream manager with per-stream health (online / degraded / offline)	Must	B	6-7	Loop 3 video files as 'cameras', one worker process each.
F09	Backend API + database + snapshot storage	Must	C	1-5	SQLite + local snapshots folder (no Postgres/object store).
F10	Supervisor dashboard: live tiles, alert feed, acknowledge / escalate / dismiss (target layout: Appendix D)	Must	C	1-9	Streamlit dashboard if React becomes a time sink.
F11	Confidence score on every alert and on live boxes	Must	C	5	Show numeric score only, no colouring.
F12	Edge optimisation: ONNX export + measured FPS per stream	Must	A	7-8	Lower input size + frame skipping on CPU; report honest numbers.
F13	Fire two-stage verification + growth tracking + look-alike suppression	Should	A+B	7-9	Only N-of-M temporal confirmation.
F14	Analytics: violations by zone / shift / PPE type + heatmap	Should	C	7-9	Simple bar charts from SQL queries.
F15	Compliance report export (CSV and PDF) per zone or shift	Should	C	8-9	CSV only.
F16	Escalation: unacknowledged after X minutes -> next level, with Telegram/webhook notification	Should	C	7-8	On-screen escalation banner + log; notification shown as a mock.
F17	Offline-first alert buffering and later sync	Should	B	10	Describe in architecture as designed-not-built (say so honestly).
F18	Failure-case gallery (before/after screenshots)	Should	A	8-10	Even 6-8 well-explained cases are enough.
F19	Badge/RFID worker ID (mock mapping, no facial recognition)	Wow	B	10	CSV mapping of track ID to a worker ID for the demo.
F20	LLM layer: natural-language questions over alerts + shift summary	Wow	C	10-11	Templated summary without an LLM; mention as roadmap.
F21	Cost/ROI calculator or slide	Wow	C	10-11	Static table with clearly labelled assumptions.
F22	Privacy: blur head region in stored snapshots	Wow	A	10	State in the docs that snapshots are cropped/limited and blur is on the roadmap.

Rule of thumb: a feature that is 100% working and honestly explained beats three features that half-work. Whenever a Tier 1 item is behind schedule, freeze all Tier 2/3 work until it is back on track.

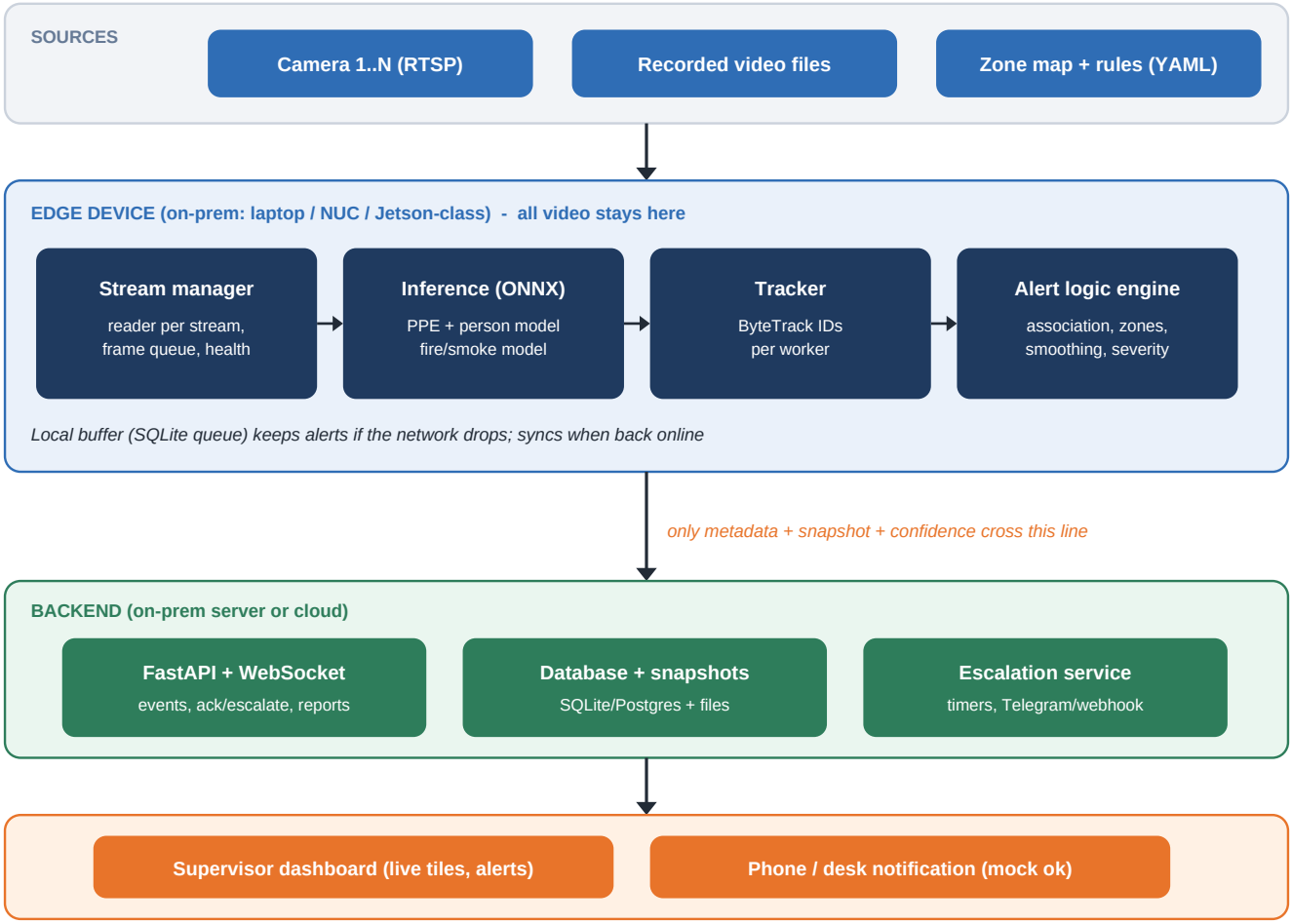
4. Do's and Don'ts

Keep this page open during the whole project. Most lost points in this kind of hackathon come from the right-hand column.

Area	DO	DON'T
Product framing	Design for the floor supervisor who uses this every shift. State the specific gap you fill.	Do not pitch 'we used AI'. Do not design only for the judges' first impression.
Model use	Fine-tune on PPE + fire/smoke data plus your own edge-case clips, and explain why your pipeline covers gaps a pretrained API does not.	Do not just call a pretrained detection API or drop in a Roboflow model and stop.
PPE logic	Report the exact missing item per person. Use an 'unverifiable' state when a person is occluded, tiny or cut off.	Do not output only 'violation detected'. Do not raise a violation when you simply cannot see the person properly.
Alerts	Use tracking, N-of-M confirmation, grace period and cooldown. Send one alert per event.	Do not alert per frame. Do not send every low-severity event as a push notification.
Severity	Make fire/smoke Critical and give it a different channel and behaviour from PPE.	Do not treat all alerts equally.
Fire / smoke	Track region growth over frames; check overlap with tracked people wearing orange/yellow; verify before Critical.	Do not trust a single-frame detection. Do not demo without testing orange vests, lamps, sunsets, red machinery.
Data	Split train/validation/test by video or scene, keep a separate hard test set (dust, dark, angled, crowded).	Do not split by random frame (near-identical frames leak into the test set). Do not test only on clean front-facing footage.
Edge	Run inference locally, export to ONNX, measure FPS and latency, state bandwidth/cost reasoning.	Do not stream raw video to a cloud service. Do not claim TensorRT/Jetson results you did not measure.
Privacy	Use badge/RFID mapping (mocked is fine), minimise stored images, optionally blur heads.	Do not use facial recognition.
Engineering	Freeze interface contracts on Day 1. Commit daily. Keep a 'demo-stable' git tag. Record a backup demo video.	Do not change the alert JSON schema late. Do not run the demo from an uncommitted laptop state.
Numbers	Report measured metrics with the test set they came from; label cost/ROI inputs as assumptions.	Do not invent accuracy, savings or incident statistics.
Scope	Finish Tier 1, then Tier 2, then Tier 3. Freeze features on Day 10.	Do not add features in the last two days. Do not start React polish before the pipeline works end to end.
Honesty	Include a 'what is mocked / simplified' slide.	Do not present mocked parts (RFID, escalation SMS, cost figures) as production-real.

5. System architecture

Everything that touches video stays on the edge device. Only small structured events and thumbnails cross to the backend and dashboard. For the hackathon, the edge device, backend and dashboard can all run on one laptop, but the boundaries are kept explicit so the design is defensible as a real deployment.



Why edge-first (use in the pitch and documentation)

- **Latency:** no round trip to a remote server; an alert can appear within about a second of the frame being captured (measure it and report your real number).
- **Bandwidth:** a 1080p H.264 camera is typically in the low single-digit Mbps range (assumption). 16 cameras streaming to a cloud is tens of Mbps, continuously. An alert plus a thumbnail is a few hundred KB, only when something happens.
- **Cost:** no cloud GPU or video egress bills; one edge device can serve several cameras.
- **Reliability and privacy:** keeps working when the internet drops; raw footage never leaves the premises.

6. Technology choices and alternatives

For each decision: our recommended choice, the realistic alternatives, and when to switch. The goal is to always have a fallback that still delivers a working demo. Verify licences and package versions yourselves before relying on them; this plan is written without live access to documentation.

Decision	Recommended	Alternatives	Switch when...
Object detector	YOLOv8 / YOLO11 (Ultralytics), small or nano size, 640 px input	YOLO-NAS, RT-DETR, YOLOX, YOLOv5	You need a permissive licence for a real product (check Ultralytics licence terms, which are copyleft as far as I know) or a model is too slow on your hardware.

Decision	Recommended	Alternatives	Switch when...
PPE compliance method	Detect person + each PPE item, decide compliance by geometry (association)	(a) explicit 'no_helmet' classes; (b) crop each person and run a small classifier per body region; (c) pose keypoints to locate head/hands/feet	Association is unreliable on your data: try (a) for helmets first (most visible item). Use (c) only if time is left.
Fire / smoke	Separate detector + temporal verification (growth, flicker, person-overlap suppression)	Single combined model with PPE; pure colour/motion heuristics; pretrained fire model as-is	Training a good separate model fails: use a pretrained fire/smoke model and invest in verification.
Tracker	ByteTrack through the Supervision library	BoT-SORT, DeepSORT, hand-written IoU tracker	ID switching is severe in crowded scenes: try BoT-SORT; if the library fights you, use the IoU tracker.
Inference runtime	ONNX Runtime (CPU or CUDA), FP16 where supported	TensorRT (NVIDIA), OpenVINO (Intel CPU/iGPU), plain PyTorch	You have an NVIDIA GPU: TensorRT for extra speed. Intel-only laptop: OpenVINO. Keep PyTorch as the always-works fallback.
Stream handling	One reader thread per stream + bounded queue (drop oldest) + shared inference worker	One process per stream; GStreamer/FFmpeg pipelines; Redis/RabbitMQ between stages	One machine cannot keep up: process per stream. Skip Redis/RabbitMQ unless you genuinely split across machines.
Backend	FastAPI + WebSocket + SQLite (swap to Postgres later)	Node.js/Express + Postgres; Django	Team is stronger in Node. Postgres only if you want the 'production' story and have time.
Dashboard	React + WebSocket (if C is comfortable with React)	Streamlit; plain HTML + JS; Grafana for analytics	React is slipping past Day 5: switch to Streamlit immediately. A working Streamlit UI beats an unfinished React UI.
Live video in the browser	MJPEG stream of annotated frames from the API	WebRTC; HLS; periodic snapshots (1-2 per second)	MJPEG lags or eats CPU: fall back to snapshots per second per tile.
Storage	SQLite + snapshots on disk	Postgres + MinIO/S3	Only when you want to show a production architecture. Not needed for the demo.
Training compute	Free notebooks (Colab/Kaggle) or any local GPU	Roboflow hosted training; college lab GPU; cloud credits	Free-tier limits interrupt long runs: use checkpoints, smaller model, fewer epochs.
Notifications	Telegram bot or generic webhook	Email (SMTP), Slack webhook, SMS trial, WhatsApp Business API	WhatsApp/SMS need approvals or paid accounts: show a mocked message card instead.
Labelling	Auto-label with a pretrained model, then correct in Roboflow or CVAT	Label Studio; fully manual	Model pre-labels are poor for a class (e.g. gloves): label that class manually and keep it small.
LLM add-on (Wow)	Claude/GPT API for questions over the alert table and shift summary	Templated summary without an LLM; local small model	No API key/budget or unreliable internet at the venue: templated summary.

7. Detailed module design

7.1 PPE-to-person association

The detector outputs boxes for person and each PPE item. For each tracked person we test whether an item box falls in the expected body region. Percentages are starting values to tune on your validation clips.

PPE item	Expected region (of the person box)	Rule (starting value)	Notes
Helmet	Top 30% (head)	Helmet box centre inside the head region AND overlap with head region above a threshold	Most reliable item. Hi-vis caps or hoods can confuse it.
Vest / hi-vis	Torso, about 20%-65% of height	Vest box overlaps torso region by more than half of the vest box	Orange vest is also a fire look-alike (see 7.4).

PPE item	Expected region (of the person box)	Rule (starting value)	Notes
Boots	Bottom 20% (feet)	Boots box centre inside the feet region	Often occluded by machinery; use 'unverifiable' liberally.
Gloves	Around arms and hands, mid to lower box	Glove box centre inside the person box (expanded ~10%) and lower 60%	Small and hard. Report as lower confidence; make it zone-dependent.
Goggles / mask (optional)	Head region	Box centre inside head region	Only if your dataset has good labels.

Three states per item, not two: **compliant** (item found), **missing** (person clearly visible, item absent), and **unverifiable** (person tiny, cut off by the frame edge, heavily occluded, or blurred). Only missing can create an alert. Unverifiable is shown in the dashboard as a grey status and counted in the report; this removes a large class of false positives and is a good talking point.

7.2 Per-worker violation state machine

For every (track_id, ppe_item):

```
COMPLIANT --(item missing in some frames)--> SUSPECTED
SUSPECTED --(missing in >= N of last M frames)--> PENDING (grace timer starts)
PENDING --(still missing after grace period)--> ALERTED (one alert created)
PENDING --(item seen again)--> COMPLIANT (no alert, timer cancelled)
ALERTED --(cooldown running: same track+item+zone)--> no new alert
ALERTED --(item compliant continuously for ~2 s)--> RESOLVED (event closed, duration stored)
```

Starting values (tune on validation clips): M = 15 processed frames, N = 10, grace = 3 s, cooldown = 60 s, resolve = 2 s. Critical events (fire) use a shorter grace period.

If the tracker loses and re-acquires a worker (ID switch), the cooldown is keyed on zone + item + approximate location as well as track ID, so a brief ID switch does not fire a duplicate alert.

7.3 Severity engine

Event	Base severity	Escalation modifiers	Response
Fire confirmed	Critical	Growing region, or confirmed on 2 cameras, or people in zone	Immediate on-screen alarm + notification to safety officer; snapshot attached.
Smoke confirmed	Critical	Growth rate above threshold; dense region	Same as fire. Unconfirmed smoke stays High until verified.
Missing helmet in active / high-risk zone	High	+1 level if 2 or more missing items on the same worker	Real-time alert to zone supervisor.
Missing gloves / vest / boots in a lower-risk zone	Medium	+1 level if repeated by the same track within X minutes	Logged; included in periodic digest.
Minor: unverifiable, low-confidence, marginal cases	Low	None	Dashboard log only; no notification.

Severity is computed from event type and the zone's risk level from the YAML config, so the same missing glove can be Medium in the warehouse and High in the welding bay.

7.4 Fire and smoke pipeline (with the false-positive fix)

- **Stage 1 - detect:** fire/smoke detector runs on every stream at a lower rate than PPE (fire changes slowly, PPE decisions need more frames).
- **Stage 2 - candidate tracking:** track the fire/smoke region over consecutive frames; compute area growth and flicker (rapid intensity/shape change).
- **Stage 3 - look-alike suppression:** if the region mostly overlaps a tracked person wearing an orange/yellow vest or jacket, or is static and uniformly coloured (a painted machine, a lamp, a traffic cone), it is suppressed or downgraded.
- **Stage 4 - confirm and grade:** N-of-M confirmation. Static and small: High. Growing or persistent: Critical.

- **Documented case:** a worker in a bright orange vest walks across the frame and no fire alert is raised. Keep the clip, the detector's raw output and the suppressed result for the failure-case gallery.

7.5 Multi-stream and load handling

- One reader per stream with a small bounded queue that drops the oldest frame (better to be current than complete).
- Health states: ONLINE (frames arriving, frame age under about 2 s), DEGRADED (reconnecting, or effective FPS below half of target), OFFLINE (no frames for about 10 s). Automatic reconnect with backoff.
- Load-aware: if the inference backlog grows, lower the processing FPS for streams in low-risk zones first; keep fire/smoke checks on every stream.
- Isolation: a crashed or frozen stream must never stop the others (separate threads/processes and timeouts).
- Demo trick: loop several recorded clips as 'cameras'. Unplug or stop one live to show the OFFLINE status and recovery.

7.6 Data model and interfaces (freeze on Day 1)

Table	Main columns
cameras	camera_id, name, source (rtsp/file), zone_id, status, last_frame_at, fps
zones	zone_id, name, polygon (per camera), required_ppe (list), risk_level
events	event_id, type (ppe_missing / fire / smoke), item, severity, camera_id, zone_id, track_id, worker_id (nullable), confidence, snapshot_path, started_at, ended_at, status (open / acknowledged / escalated / dismissed / resolved)
event_actions	action_id, event_id, action (ack / escalate / dismiss / auto_escalate), by_user, at, note
worker_map (optional)	track_id or badge_id, worker_id, shift
outbox (edge buffer)	payload_json, created_at, sent (bool)

Alert JSON (contract between edge and backend):

```
{
 "event_id": "evt_000123",
 "type": "ppe_missing",              // ppe_missing | fire | smoke
 "items_missing": ["helmet", "gloves"],
 "severity": "High",                 // Critical | High | Medium | Low
 "camera_id": "cam_02",
 "zone_id": "welding_bay",
 "track_id": 17,
 "confidence": 0.86,
 "timestamp": "2026-09-20T10:42:07Z",
 "snapshot": "snapshots/evt_000123.jpg",
 "bbox": [412, 118, 590, 470]
}
```

Key API endpoints: POST /events (edge -> backend), GET /events?zone&severity&status, POST /events/{id}/ack, /escalate, /dismiss, GET /cameras (status), GET /stream/{camera_id} (MJPEG), GET /analytics/summary, GET /reports/export?zone&shift&format, WebSocket /ws/alerts (live push).

Zone rules config (example)

```
cam_02:
  zones:
    welding_bay:
      polygon: [[100,200],[620,200],[620,460],[100,460]]
      required_ppe: [helmet, gloves, goggles, vest]
      risk: high
    walkway:
      polygon: [[620,200],[900,200],[900,460],[620,460]]
      required_ppe: [helmet, vest]
      risk: medium
```

8. Data and model plan

8.1 Classes

PPE model: person, helmet, vest, boots, gloves (optional: goggles, mask). Fire model: fire, smoke. Keep the classes few and well labelled: with a small team, a small clean set beats a large messy one.

8.2 Data sources

- Public datasets: PPE sets on Roboflow Universe and Kaggle; fire/smoke sets such as D-Fire and FASDD (both named in your strategy report). Check each dataset's licence, class names and annotation quality before merging; harmonise class names.
- Own edge-case clips: record or find short clips for every scenario in section 9 (dust, steam/haze, low light, backlit, high oblique angle, crowded/occluded, orange clothing, red/orange machinery, lamps). Mixed sources are fine; state their origin in the docs.
- Negatives: include images with no fire but fire-coloured objects, and workers correctly wearing everything, so the model learns what not to flag.

8.3 Labelling with a small team

- Use a pretrained model to pre-label, then correct boxes. Budget: a few hundred own images, prioritising hard cases over easy ones.
- Write a one-page labelling guide (what counts as a helmet, when to label a partially visible glove, minimum box size) so everyone labels consistently.
- Extract frames every 1-2 seconds from each clip instead of every frame: consecutive frames are near-duplicates and waste labelling time.

8.4 Splits and leakage

Split by video/scene, never by random frame. If frames from the same clip land in both train and test, your metrics will look great and the demo will disappoint. Keep a small hard test set (dust, dark, angled, crowded, look-alikes) that is never used for training.

8.5 Augmentation and training recipe

- **Augmentation:** brightness/contrast/gamma jitter (night, backlit), blur and noise, synthetic haze/fog/dust overlay, random occlusion patches, perspective/rotation for oblique angles, scale changes for distant workers, mosaic (default in YOLO training).
- **Training:** start from pretrained weights (small/nano), 640 px, fine-tune first on public data, then on public + own data. Compare versions (v1 baseline, v2 with edge cases) on the same held-out sets and keep the table.
- **Evaluation:** per-class precision/recall and mAP on the normal test set AND the hard test set; plus system-level metrics (section 12). Review every top false positive and false negative by eye.

8.6 Model outputs handed to Person B (contract)

A detection = {class, confidence, bbox, frame_id, camera_id}. Person A publishes a mock detections file on Day 1 so that B and C can build the whole pipeline before the first real model exists.

9. Robustness plan and failure-case gallery

Each row is a scenario to record, test and screenshot. The gallery (F18) shows for each: the tricky input, what a naive system would do, what ours does, and why.

Scenario	Risk	Mitigation in our system	How to test
Dust / steam / haze	Missed detections, false smoke alarms from steam	Haze augmentation; smoke verification needs growth/persistence; steam near a vent zone can be down-weighted	Clips with steam/dust; check missed workers and false smoke
Low light / night shift	Noisy image, missed vests and helmets	Gamma/brightness augmentation; per-class confidence thresholds	Darkened and real night clips
Backlit / overexposed	Silhouettes, washed-out colours	Overexposure augmentation; 'unverifiable' state for silhouettes	Clips facing a window or lamp
High-mounted / oblique angle	Small, foreshortened people	Oblique-angle training data; region rules relative to person box	Clips from above at 30-60 degrees
Occlusion / overlapping workers	Duplicate or missed alerts	Tracker with re-ID tolerance; 'unverifiable' instead of 'missing'; cooldown keyed on zone+item+location	Crowded walkway clips
Orange / yellow clothing, cones	Fire false positive	Person-overlap suppression, static-colour check, growth/flicker confirmation	Worker in orange vest walking through frame (the documented case)
Fire-coloured machinery, lamps, sunset	Fire false positive	Colour + motion + persistence checks; look-alike negatives in training	Clips with red/orange objects and lights
Hi-vis cap vs helmet	Helmet false negative or positive	Include such samples in training; report ambiguity via confidence	Workers in caps/hoods
Camera freeze / disconnect	Silent failure	Health monitor (frame age), auto reconnect, OFFLINE badge	Stop a stream during the demo
Network drop	Lost alerts	Local outbox queue + sync (F17)	Disable backend, generate alerts, restore

10. Team structure and ways of working

10.1 Roles and ownership

	Person A - ML / Vision lead	Person B - Pipeline and alert logic	Person C - Backend and dashboard
Owns	Datasets, labelling, PPE model, fire/smoke model, augmentation, ONNX export, FPS benchmarks, evaluation tables, failure-case gallery	Stream manager, tracker, PPE-to-person association, zone rules, state machine, severity engine, fire verification logic, offline buffer	FastAPI, database, snapshot storage, WebSocket, dashboard, acknowledge/escalate/dismiss, analytics, reports, notifier
Best fit for	Someone comfortable with Python, PyTorch/Ultralytics and data wrangling	Strong Python/OpenCV and logic/debugging skills	Full-stack (Python API + React or Streamlit). If C is frontend-only, B also builds the API.
Delivers	Model files + mock detections (Day 1), model card with metrics	Engine that turns detections into alerts, config files	Running API + dashboard against mock alerts (Day 2)
Also leads	Model/evaluation part of the documentation and the gallery	Architecture and edge-justification part of the documentation	Pitch deck, cost/ROI slide, demo script

Shared: integration, stress testing, final documentation and demo rehearsal are done by everyone. Each person has a named backup for the others' area (A backs up B, B backs up C, C backs up A) so one absence does not stall the project.

10.2 Interfaces frozen on Day 1

- **Detection format (A to B):** class, confidence, bbox, frame_id, camera_id. A ships a mock detections JSON on Day 1.
- **Alert JSON (B to C):** section 7.6. Changes only by agreement of the whole team, with a version field.
- **Zone/rules YAML (B and C):** section 7.6.
- **Camera registry (B and C):** camera_id, source, zone mapping.
- **Health/status format (B to C):** per-stream state, FPS, last frame time.

10.3 Ways of working

- **Daily 15-minute stand-up:** what I finished, what I do today, what blocks me. Post one line in the group chat if you cannot attend.
- **Git:** main always runs; work on short feature branches (feat/tracker, feat/dashboard); merge by pull request reviewed by one other person. Tag v0.5-m1, v0.9-m2 and demo-stable.
- **Shared drive** for datasets, model versions (v1, v2 ... with a metrics note) and test clips. Never overwrite a model file; add a new version.
- **Definition of done:** works on the shared demo clips, has a config not hard-coded values, has a short README note, merged to main.
- **Decision log:** one shared page listing each decision (e.g. 'React vs Streamlit'), date and reason. Prevents re-arguing later.
- **Time-box rule:** if any task takes more than twice its estimate, switch to its Plan B and tell the others the same day.
- **Working hours:** the plan assumes about 5-6 focused hours per person per day. If real availability is lower, cut Tier 2 first (section 11.3).

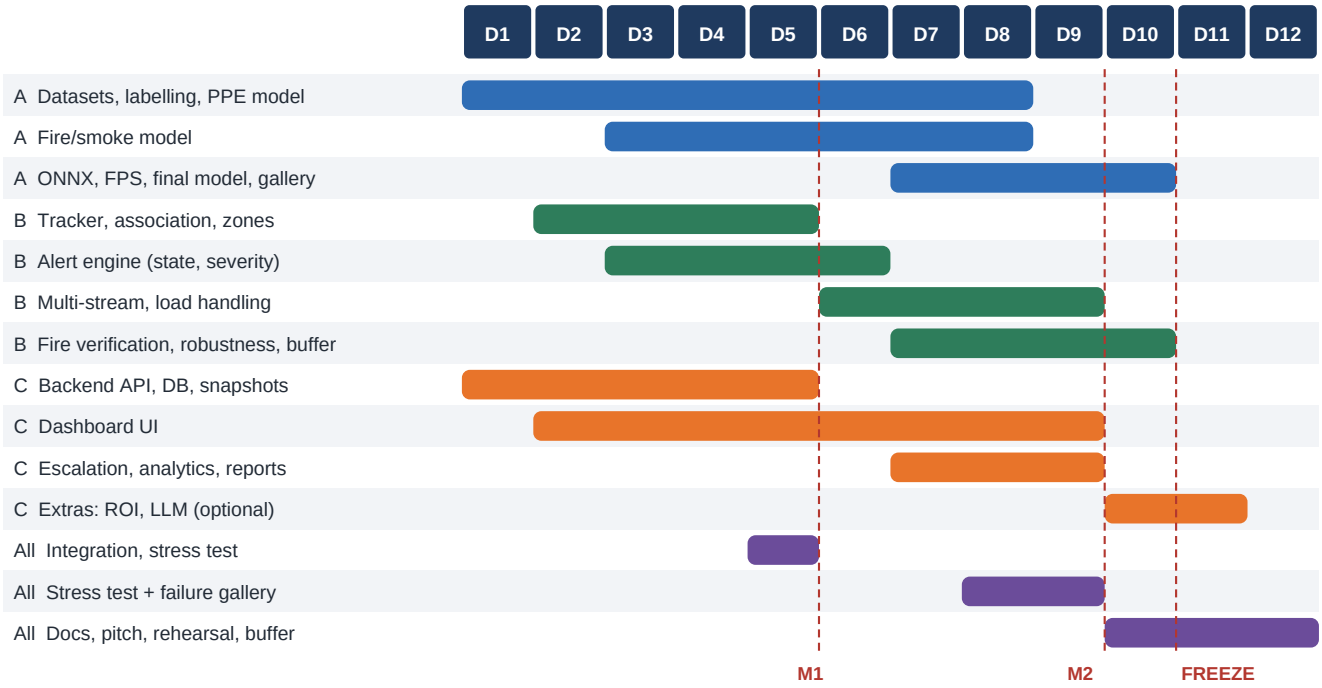
10.4 Shared support tasks (open to everyone)

Any team member can pick up the low-coordination tasks below at any time. They speed the whole project up and free the workstream owners to stay on the critical path.

- Find and record clips for the robustness scenarios (section 9) and note the source/licence of each.
- Draw bounding-box corrections in Roboflow/CVAT once A has written the labelling guide.
- Test the demo as a 'supervisor': write down every point where the dashboard confuses them.
- Proof-read documentation and help write the pitch story and cost/ROI assumptions.

11. Schedule (12 days, with a 10-day version)

Days are working days from your kick-off. Milestone **M1** (end of Day 5) = one stream end to end. Milestone **M2** (end of Day 9) = Tier 1 + Tier 2 complete or consciously dropped. **Feature freeze** at the end of Day 10.



Colour key: blue = Person A, green = Person B, orange = Person C, purple = all. Dashed lines: M1 (after Day 5), M2 (after Day 9), feature freeze (after Day 10).

11.1 Day-by-day plan

Day	Person A (ML)	Person B (pipeline / logic)	Person C (backend / UI)	Exit criterion
1	Study datasets; download PPE + fire/smoke sets; set up training environment; run a pretrained YOLO on sample clips; publish mock detections file.	Repo, environment, folder structure; freeze detection and alert contracts with A and C; read frames from a video file with OpenCV.	Scaffold API + dashboard; DB schema; choose React vs Streamlit (decide today); mock alert endpoint.	Everyone can run the repo; contracts v1 frozen; 4-5 test clips collected.
2	Clean and merge datasets; list own-footage scenarios; label first 100-150 own images; first PPE fine-tune.	Integrate ByteTrack on person detections; zone polygon loader from YAML; stream reader with reconnect.	Alert feed with mock data; camera tile grid; WebSocket push; write events to DB.	Mock alerts appear live on the dashboard.
3	Evaluate PPE baseline per class; start fire/smoke baseline; error analysis.	PPE-to-person association v1; per-worker state machine v1 (mock or baseline detections).	Alert card (snapshot, zone, camera, severity, confidence); ack / escalate / dismiss endpoints and buttons.	One clip -> per-person missing items printed and stored.
4	Fire/smoke baseline eval; collect look-alike negatives; PPE v1.5 with own data.	Zone rules engine wired to association; severity engine v1; dedup + cooldown.	Live annotated tiles (MJPEG or snapshots); stream status badges; snapshot storage.	Zone-aware alerts from one stream reach the dashboard (rough).
5	Export ONNX v1 and run inside the pipeline. Fix data issues found in integration.	Tune smoothing (N, M, grace) on real clips; unverifiable state; bug fixing.	Dashboard polish; confidence display; filters; snapshot links.	M1: one stream, PPE alerts with severity and ack workflow, screen-recorded. Tag v0.5-m1.
6	Integrate fire model v1; start haze/low-light augmentation; retrain PPE v2 in the background.	Multi-stream manager; per-stream config; health monitor; frame dropping.	Multi-tile layout; per-stream status; alert filters (zone, severity, camera).	3 streams run simultaneously with alerts.
7	ONNX FP16/INT8 tests; FPS table per stream and hardware; evaluate PPE v2.	Fire verification (growth, flicker, person-overlap suppression) with A; load-aware frame skipping.	Escalation timer + Telegram/webhook notification; analytics v1 (by zone/shift).	FPS table exists; fire alert reaches Critical and triggers a notification.

Day	Person A (ML)	Person B (pipeline / logic)	Person C (backend / UI)	Exit criterion
8	Run the hard test set (dust, dark, angled, occluded); list top 10 failures; fine-tune on them.	Threshold tuning per class; occlusion handling; false-positive suppression review.	CSV/PDF report export; heatmap; shift/zone filters.	Hard-set results table v1 and failure list.
9	Failure-case gallery (screenshots + explanation); model v3 if needed.	Soak test (30+ min, several streams); kill a stream; kill network; fix bugs.	UI hardening; supervisor UX pass; empty/error states.	M2: Tier 1 + Tier 2 done or dropped. Tag v0.9-m2.
10	Final model selection; model card with metrics; optional head blur.	Offline buffer (F17); optional badge/worker mapping; architecture + edge-justification doc.	Optional LLM query/summary; cost/ROI slide; docs skeleton.	Feature freeze. Only fixes after today.
11	Documentation for models/data; gallery final; Q&A prep.	Documentation for pipeline/architecture; final soak; demo config.	Pitch deck; demo script; record backup demo video.	Docs and slides complete; 2 rehearsals done.
12	Buffer: final bug fixes only.	Fresh-machine install test; freeze demo-stable.	Final rehearsal; submission package.	Submitted. No new features.

11.2 If you only have 10 days

- Merge Days 1 and 2 (foundations + data) into one longer day; run A's labelling in parallel with B's and C's scaffolding.
- Drop Day 10 extras except F17 (offline buffer) if cheap; move docs and pitch earlier so they overlap with Days 8-9.
- Use the buffer day (12) as Day 10: keep only bug fixes, rehearsal and submission.
- Everything else remains; if it is still too tight, apply the cut order below.

11.3 Cut order when time runs short

Cut first	Then	Never cut
F20 LLM layer, F22 head blur, F19 RFID mock, F21 calculator (keep as a table)	F17 offline buffer (describe only), F15 PDF export (keep CSV), F14 heatmap (keep bar charts), F16 real notification (keep mock)	F01-F12: per-item PPE, tracking, zones, severity, multi-stream, dashboard, edge justification, one documented false-positive case

12. Testing and evaluation plan

12.1 Metrics to report (measured, not assumed)

Metric	How to measure	Working target (to aim for, not a promise)
Precision / recall per class	Held-out test set and hard test set, reported separately	Improve v1 -> v2 -> v3; show the table
False alerts per camera per hour	Run a long clip of compliant workers; count alerts before and after smoothing	Near zero on demo clips; show before/after
Duplicate alert rate	Count repeated alerts for the same worker/event	Near zero with tracking + cooldown
End-to-end alert latency	Timestamp at frame capture vs alert visible on dashboard	About 1-2 s on demo hardware; report your real number
Throughput (FPS per stream, streams per device)	Benchmark PyTorch vs ONNX (and FP16/INT8 if used) on your hardware	Enough for smooth PPE decisions at your processing rate; report honestly
Fire look-alike false alarms	Set of orange vest / lamp / machinery clips	Zero fire alerts on the look-alike set
Stream recovery	Kill/restart a stream and the network	Status changes to OFFLINE/DEGRADED and recovers automatically

12.2 Test cases checklist

Test	Expected behaviour
Worker with no helmet enters a zone requiring helmets	After the grace period, one High alert with correct zone, camera, item and confidence
Same worker stays without helmet for 5 minutes	No alert spam; event stays open with duration
Worker adds helmet before the grace period ends	No alert
Two workers overlap; one occludes the other's feet	No false 'boots missing'; status shown as unverifiable
Worker without gloves in the warehouse vs in the welding bay	Medium or none in the warehouse (per config); High in the welding bay
Worker in orange vest crosses the frame	No fire alert (documented case)
Real fire/smoke clip	Critical alert; severity increases with growth; notification sent
Dark and dusty clips	Detections continue or state becomes unverifiable; report metrics
One stream stops during the demo	OFFLINE badge, other streams unaffected, recovers automatically
Supervisor acknowledges, escalates, dismisses	Status and timestamp stored; dismissed events feed the false-positive log
Unacknowledged alert past the timer	Auto-escalation banner + notification

13. Deployment, bandwidth and cost angle

13.1 Hardware options for the edge device

Option	Runtime	Good for	Trade-off
Laptop / desktop with NVIDIA GPU	ONNX Runtime CUDA or TensorRT	Hackathon demo, several streams	Not an industrial form factor
Intel NUC-class mini PC	OpenVINO or ONNX Runtime CPU	Realistic on-prem box in a plant	Fewer streams per device without a dedicated GPU
NVIDIA Jetson-class module	TensorRT	Compact, low-power edge deployment	Extra setup effort; only claim it if you actually test on one
On-prem server	GPU + ONNX/TensorRT	Many cameras in one plant	Higher upfront cost

13.2 Bandwidth reasoning (present as assumptions)

Assume one 1080p H.264 camera at roughly 2-6 Mbps (typical range; use your own camera's bitrate if known). For 16 cameras that is roughly 32-96 Mbps of continuous upload if video goes to a cloud, around the clock. In the edge design, only alerts leave the device: for example 50 alerts per day at about 200 KB per snapshot is about 10 MB per day. Show this calculation with your own numbers on one slide.

13.3 Cost and ROI framework

Do not quote invented savings. Give a formula and a table of clearly labelled assumptions that a judge can change.

Item	Formula / what to fill in
One-time cost	Edge device(s) + installation + camera additions (most plants already have CCTV) + integration effort
Yearly running cost	Maintenance + model updates + electricity + notification services
Yearly benefit (assumption-based)	(incidents avoided x average cost per incident) + insurance premium reduction + supervisor time saved on manual inspection + audit-preparation time saved
Payback period	One-time cost / (yearly benefit - yearly running cost)
Sensitivity	Show the result for a pessimistic, expected and optimistic set of assumptions

14. Privacy, ethics and honest limitations

- **No facial recognition.** Worker accountability uses a badge/RFID mapping (mocked in the prototype).
- **Data minimisation:** store thumbnails and snapshots for events only, with a retention period; no continuous raw video storage. Optional head blur.
- **Human in the loop:** the system assists supervisors; it does not discipline anyone. Show confidence and let people dismiss borderline cases.
- **Bias and fairness:** test across skin tones, body sizes, clothing styles and PPE colours in your data; report gaps you find.
- **Limitations to state openly:** gloves and small items are hard at long range; performance depends on camera placement and resolution; smoke in steamy areas needs zone-specific tuning; the prototype's notifications, RFID mapping and cost figures are mocked or assumption-based; results come from a small custom test set.

15. Risk register

Risk	Likelihood / impact	Early warning	Mitigation	Plan B
Gloves/boots detection too weak	High / Medium	Low recall on val set by Day 4	More own data; make item zone-dependent; unverifiable state	Report as advisory; focus on helmet + vest
Fire model false alarms on orange clothing/lamps	High / High	Look-alike clips trigger alerts	Look-alike negatives; person-overlap suppression; growth check	Higher N-of-M; downgrade single-frame to High
ID switching in crowds	Medium / Medium	Duplicate alerts in crowded clip	Cooldown keyed on zone+item+location; try BoT-SORT	Zone-level alerts instead of per-worker
Too slow with several streams	Medium / High	FPS drops in Day 6-7 tests	ONNX FP16/INT8, lower resolution, frame skipping, fire at lower rate	Fewer streams in the demo; show benchmark honestly
Dashboard falls behind	Medium / High	No live feed by Day 4	Decide React vs Streamlit on Day 1; time-box	Switch to Streamlit
Training compute or data delays	Medium / High	First model not ready by Day 3	Start from pretrained weights; smaller model	Use public pretrained weights + own logic layers
Team member unavailable	Medium / High	Missed stand-ups	Named backups; frozen contracts; docs as you go	Cut order (section 11.3)
Live demo failure (Wi-Fi, laptop, GPU)	Medium / Very high	-	Everything local; pre-tested clips; demo-stable tag	Recorded backup video; second laptop
Scope creep in the last days	High / High	New features after Day 10	Feature freeze; decision log	Revert to demo-stable tag
Licence or data-source problems	Low / Medium	Unclear dataset licence	Check licences on Day 1; log sources	Swap dataset; use own clips

16. Demo and pitch plan

16.1 Rehearsed 90-second live demo

1. Dashboard with 3-4 live tiles and status badges (one stream deliberately stopped, showing OFFLINE, then recovering).
2. A worker without a helmet enters the welding bay: after the grace period a High alert appears with zone, camera, item, confidence, snapshot.
3. Acknowledge the alert; show the stored status and time. Show that staying non-compliant does not create more alerts.
4. Worker in an orange vest walks through: no fire alert. Flash the gallery card explaining why.
5. Play the smoke/fire clip: Critical alert, severity rises as the region grows, mock phone notification arrives.
6. Show a dusty/dark clip and the metrics table (normal vs hard test set).

7. Analytics and exported report; then the edge-vs-cloud bandwidth/cost slide.

16.2 Slide outline (10 slides)

#	Slide	Content
1	The problem	Real pain: delayed detection, alert fatigue, no context. One concrete factory scenario.
2	The gap	What binary, cloud-based, single-camera tools miss.
3	Our solution	Per-item PPE + zone rules + severity + fire verification + edge-first, in one diagram.
4	Architecture	Edge-to-dashboard diagram; what crosses the boundary.
5	Live demo	Rehearsed click path.
6	Robustness	Failure-case gallery highlights, including the orange-vest case.
7	Results	Measured metrics (normal and hard test sets), FPS, latency, false alerts before/after smoothing.
8	Business case	Bandwidth, hardware, ROI framework with labelled assumptions.
9	Honest limitations	What is mocked or simplified; what production would need.
10	Roadmap	RFID integration, thermal cameras, more PPE types, on-site pilot.

16.3 Questions judges are likely to ask

Question	Prepared answer (adapt with your real numbers)
Why not just use a pretrained PPE API?	It gives binary, contextless output. Ours adds per-item association, zone rules, tracking, severity and fire verification, and runs locally.
How do you handle false positives?	N-of-M confirmation, grace period, cooldown, unverifiable state, fire look-alike suppression, confidence shown to the supervisor. Show before/after numbers.
What if a worker is partly hidden?	The item status becomes unverifiable instead of missing; no alert is raised for it.
Why edge instead of cloud?	Latency, bandwidth, cost, privacy and resilience. Show the calculation.
How does it scale to 50 cameras?	One stream worker per camera, load-aware frame skipping, multiple edge devices per site. State what you tested (e.g. 3-4 streams) and what is estimated.
How accurate is it?	Give measured precision/recall on the normal and hard test sets, and say how the data was split.
What about worker privacy?	No facial recognition; badge-based ID; snapshots only for events; optional head blur.
What did you not build?	Answer directly using your honest limitations slide.

17. Judging checklist mapping

Use this table on Day 9 and Day 11 to check that every judging criterion from your documentation has visible evidence.

Judging criterion	Where we satisfy it	Evidence to show
Detects specific PPE items, not just binary violation	F01, F02	Alert card listing missing items per worker
Working severity/priority system (fire is not a missing glove)	F07, section 7.3	Critical fire alert vs Medium glove alert in the demo
Dashboard shows location, type, severity, timestamp per alert	F10 (layout: Appendix D)	Alert cards and feed
Robustness against dust/steam/lighting/angle	Section 9, F18	Hard-set metrics + failure-case gallery
At least one documented false-positive corner case	F13, section 7.4	Orange vest clip and gallery card
Supports multiple simultaneous streams	F08	3-4 tiles running with health badges

Judging criterion	Where we satisfy it	Evidence to show
Edge/local inference with bandwidth/latency justification	F12, section 5, 13	FPS/latency table and bandwidth slide
Cost and deployment feasibility angle	Section 13, F21	ROI framework slide with assumptions
Clear articulation of the real industry gap	Slides 1-3	Problem/gap slides and pitch

Appendix A. Suggested repository structure

```
ppe-fire-safety-ai/
  edge/
    stream_manager.py    # readers, queues, health, reconnect
    inference.py         # ONNX loading, batching, pre/post-processing
    tracker.py           # ByteTrack wrapper
    association.py       # PPE-to-person geometry rules
    state_machine.py     # N-of-M, grace, cooldown, resolve
    severity.py          # severity + zone risk
    fire_verify.py       # growth, flicker, look-alike suppression
    outbox.py            # offline buffer + sync
    config/zones.yaml
    config/cameras.yaml
  backend/
    main.py              # FastAPI app, WebSocket
    models.py  db.py     # tables, queries
    escalation.py        # timers, Telegram/webhook
    reports.py           # CSV/PDF, analytics
  dashboard/             # React or Streamlit app
  training/
    datasets/  notebooks/  configs/  labelling_guide.md
  eval/                  # metrics scripts, hard test set results
  docs/
    architecture.md  model_card.md  failure_gallery/  cost_roi.md  decision_log.md
  demo/
    clips/  script.md  backup_video.mp4
```

Appendix B. Day-1 checklist

- Repository created; everyone can clone, install and run a hello-world of their own module.
- Detection format, alert JSON and zones YAML written down and agreed (section 7.6, 10.2).
- Mock detections file published by A; mock alert endpoint live from C.
- React vs Streamlit decision recorded in the decision log.
- Dataset list with licences and class-name mapping started.
- Shared drive and naming convention for models, clips and screenshots set up.
- Stand-up time fixed; backups named; time-box rule agreed.

Appendix C. Things to verify yourselves

This plan was prepared without live access to documentation or dataset sites. Before relying on them, verify: dataset names, licences and class definitions (Roboflow Universe, Kaggle, D-Fire, FASDD); Ultralytics licence terms if you plan any commercial use; current Supervision/ByteTrack API details; Telegram/WhatsApp API requirements; and the exact hackathon rules on allowed pretrained models, internet use and submission format. All numeric values marked as starting values or targets should be tuned on your own data.

Appendix D. Dashboard design reference (SentinelEye)

Target look and layout for the supervisor dashboard (F10). It is a design mock-up: every number, worker ID and time shown is a placeholder to be replaced by live or measured data. Each panel carries the feature ID (F06-F15) it belongs to in section 3.

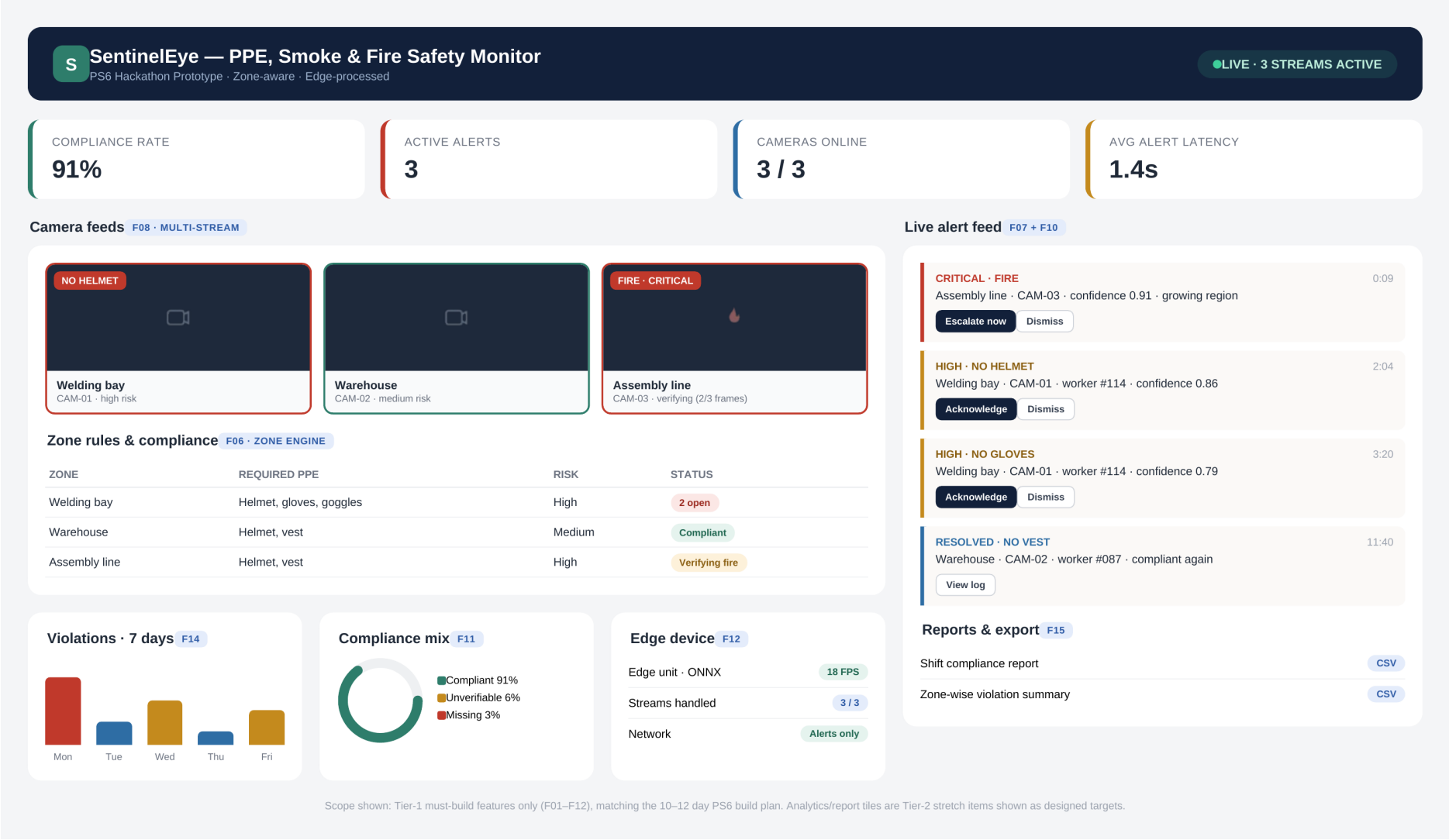


Figure D1. Target supervisor dashboard layout (SentinelEye). Values are illustrative placeholders.

How the dashboard panels map to the build

Use this table when building the dashboard so each panel is wired to a real source instead of hard-coded numbers.

Panel in the design	Feature	What it shows	Where the data comes from
Header status pill ("LIVE - 3 streams active")	F08	Whether the system is live and how many streams are running	GET /cameras (per-stream status)
KPI row: compliance rate, active alerts, cameras online, avg alert latency	F08, F10, F12	Four headline numbers for the current shift	GET /analytics/summary, GET /events?status=open, GET /cameras. Latency = frame capture to alert visible (section 12.1).
Camera feeds	F08	One tile per stream with zone, camera ID, risk level and a badge (NO HELMET, FIRE - CRITICAL, or "verifying 2/3 frames")	GET /stream/{camera_id} (MJPEG), or 1-2 snapshots per second as the fallback
Zone rules & compliance	F06	Required PPE and risk per zone, plus status (Compliant, "2 open", Verifying fire)	Zone config (section 7.6) + open events per zone
Live alert feed	F07, F10, F11	Severity-coloured cards with zone, camera, worker, confidence and time; Escalate now / Acknowledge / Dismiss / View log	WebSocket /ws/alerts; POST /events/{id}/ack, /escalate, /dismiss
Violations - 7 days	F14	Bar chart of violations per day	GET /analytics/summary
Compliance mix	F11 (tag in mock-up)	Share of compliant / unverifiable / missing decisions	GET /analytics/summary
Edge device	F12	ONNX FPS, streams handled, network mode ("Alerts only")	Measured FPS from the benchmark table; stream count from GET /cameras
Reports & export	F15	Shift compliance report and zone-wise violation summary as CSV	GET /reports/export?zone&shift&format=csv

Design notes

- **Colour language:** red = Critical, amber = High, blue = resolved / informational, green = compliant. Keep this consistent across tiles, alert cards and badges.
- **Unverifiable is its own status** (compliance mix) and is not counted as missing, in line with section 7.1.
- **Read requirements from config.** The zone table should be filled from the zones YAML, not typed into the UI. Note that the mock-up's welding-bay row lists helmet, gloves and goggles, while the YAML example in section 7.6 also includes vest; keep both in sync from the config file.
- **Build order:** camera tiles, zone status, alert feed with acknowledge / escalate / dismiss, confidence and the edge panel are Tier 1. The violations chart and report export are Tier 2 (F14, F15), as the footnote in the mock-up says.
- **If React slips** (see section 6), the same layout can be rebuilt in Streamlit with a metrics row, two columns and a table; keep the panel order and colours.